Name: Mary-Rose Tracy

ID#: 1001852753

Date: 09/28/2024

Instruction

What the question is

Already did these problems in green PDA’S AND PUMPING LEMMA

Turing Machine Basics from Decidability

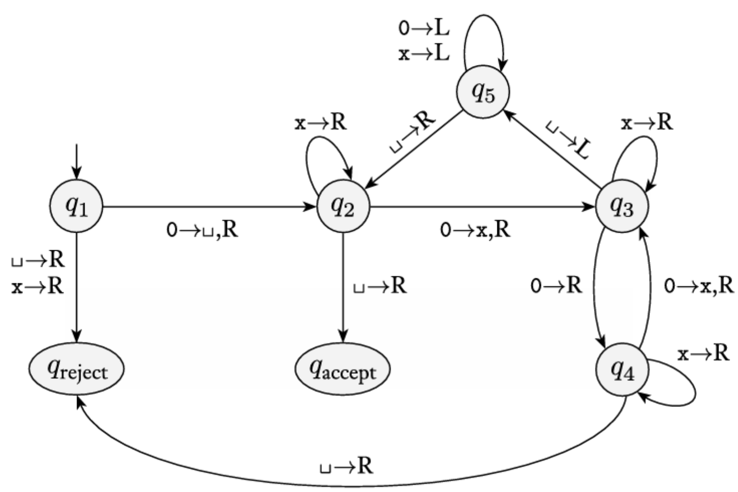
TM Configurations: 3.1b, 3.2b, 3.2c

3.1 This exercise concerns TM M_2, whose description and state diagram appear in Example 3.7. In each of the parts, give the sequence of configurations that M_2 enters when started on the indicated input string.

[redacted]

A **b.** 00.

[redacted]



3.1b Have 2 run machine M_2 on input 00. The starting config = $q_1 00$. The set of config that the machine enters when started on the input str is as follows.

	At:	→:	L or R	Goes to the
$q_1 00$	$q_1, 0$		R	q_2
$\sqcup q_2 0$	$q_2, 0$		R	q_3
$\sqcup x q_3 \sqcup$	$q_3, \sqcup$		L	q_5
$\sqcup q_5 x \sqcup$	q_5, x		L	q_5
$q_5 \sqcup x \sqcup$	q_5, x		R	q_2
$\sqcup q_2 x \sqcup$	$q_5, \sqcup$		R	accept state
$\sqcup x q_2 \sqcup$	$q_2\ x$		R	accept state
$\sqcup x \sqcup$ accept state	$q_2 \sqcup$			

~~The state q on 0 the machine goes → q_2~~

I think I don't need to over explain it.

Finally M_2 enters q accept state → & is accepted.

Double check, since I did it again:

3.1b | run machine M_2 on the input 00. Starting config is q_1 00. sequence of configs that the machine enters when started on input is....

Mach - machine
R = Right
L = Left
* = Repeated

	At $q_{\#}$	$\$_{\#}$	→	Goes To the $q_{\#}$
$q_1 00$	q_1	0	⊔,R	q_2
$⊔q_2 0$	q_2	0	X,R	q_3
$⊔Xq_3⊔$	q_3	⊔	L	q_5
$⊔q_5X⊔$	q_5	X	L	q_5
$q_5⊔X⊔$	q_5	⊔	R	q_2
$⊔q_2X⊔$	q_2	~~X~~	R	Accept state
$⊔Xq_2⊔$	q_2	⊔	R	Accept State
$⊔X⊔q_{accept}$				

q_1 on 0 mach goes to state q_2, Does ⊔ & move h to R. q_2 on 0 mach goes to state q_3, Does X, moves h to R. q_3 on ⊔ mach goes to state q_5 moves head to L. q_5 on X mach goes to state q_5 moves h to L. q_5 on ⊔, machine goest to state q_2, moves h to R. q_2 on X mach goes to state q_2 it self. Moves head R. q_2 on ⊔ mach goes goes to state q_{accept} then halts. Then M_2 enters q_{accept} state, then 00 is accepted

3.2 This exercise concerns TM M_1, whose description and state diagram appear in Example 3.9. In each of the parts, give the sequence of configurations that M_1 enters when started on the indicated input string.

A[redacted]

b. 1#1.

c. 1##1.

[redacted]

3.2b)

q_1 1#1
x q_3 #1
x # q_5 1
x q_6 # x
q_7 x # x
x q_1 # x
x # q_8 x
x # x q_8 ⊔
x # x ⊔ q_{accept}

M_1 enters q_{accept} state. Thus, input 1#1 accepts.

3.26 Lets do 1##1

Sequence of Config that M_1 enters

q_1 1##1

x q_3 ##1

x# q_5 #1

x## q_{reject} 1

[q_5 is not reading #, so it enters to reject state.

Fin M_1 goes to q reject & so 1##1 = rejected

Double check, since I did it again:

3.5 [illegible]

3.2b input str 1#1

The set of config. that M_1 enters are... (for both questions

q_1#1

xq_3#1

x#$q_5$1

xq_6#x

q_7x#x

xq_1#x

x#q_8x

x#xq_8⊔

x#x⊔q_{accept}

finally M_1 enters q_{accept}. Thus 1#1 = accepted.

3.2c 1##1

$q_1$1##1

xq_3##1

x#q_5#1 NOTE: The state q_5 ≠ reading #. Thus enters reject.

x##q_{reject}1

finally M_1 enters q_{reject}. Thus 1##1 = rejected.

TM trivia: 3.5

[A]**3.5** Examine the formal definition of a Turing machine to answer the following questions, and explain your reasoning.

a. Can a Turing machine ever write the blank symbol $\sqcup$ on its tape?

b. Can the tape alphabet Γ be the same as the input alphabet Σ?

c. Can a Turing machine's head *ever* be in the same location in two successive steps?

d. Can a Turing machine contain just a single state?

3.5 A) yes - T.M can write blank sym on its tape

according to the def. A T.M is a 7 tuple $(Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$

where Q, Σ, Γ = finite

Σ is the input alphabet not containing the blank sym.

Γ is the tape alphabet, A T.M can write any char. in Γ on it's tape. T.M can write blank sym on its tape.

B) No, The Tape alphabet Γ never same as input alphabet doesn't contain blank sym. Always Σ is subset of Γ, but never same as Σ.

C) yes. T.M head be in the same location in 2 successive steps. When the T.M head ← left of the end of tape. If T.M again try to move ← left side then T.M head in same locat as the prev moves.

D) No,

They are distinct & independent

A T.M is a 7 tuple $(Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$

Q = set of states

q_{accept} & q_{rej} = belongs Q but they don't = eachother

T.M contains at least 2 states.

Double check, since I did it again:

$\boxed{3.5\ a}$ yes, A T.M can write the blank sym ⊔ on its

$\boxed{3.5\ b}$ NO,

$\boxed{3.5C}$ yes,

$\boxed{3.5D}$ NO,

TM Closure: 3.16d, 3.16b (Use Nondeterminism), 3.16c (Use Nondeterminism)

3.16 Show that the collection of Turing-recognizable languages is closed under the operation of

[redacted]

b. concatenation.

c. star.

d. intersection.

[redacted]

3.16 b Suppose x & y be 2 ~~Turing Machines that~~ recognizable lang that have Turing Machines M_x & M_y respectively. Concat these lang is called by L_{xy} & the Turing machine recognize Lang as M_{xy}. It / each str of xy into s_1 & s_2 non-deterministically runs $s_1 \rightarrow M_x$. If M_x halts & rejects → reject. runs $s_2 \rightarrow M_y$. If M_y accepts → accept. If M_y halts → reject or rej.

Summary:

thus, Turing recog. Lang. is closed concat.

3.16 c Star

x be Turing recog. Lang & the T.M = M_x. X^* is the Lang obtained from Star operation on x, T.M for Lang be M_{x^*}

M_{x^*}: For an input str s of X, it non-deterministically / the str into $s_1, s_2, ..., s_n$.

For each of the / Parts M_{x^*} = runs. M_{x^*} %ed Pts then s = accepte. else s = rejected from M_{x^*}

Conclusion:

hence Turing recog Lang = close under star.

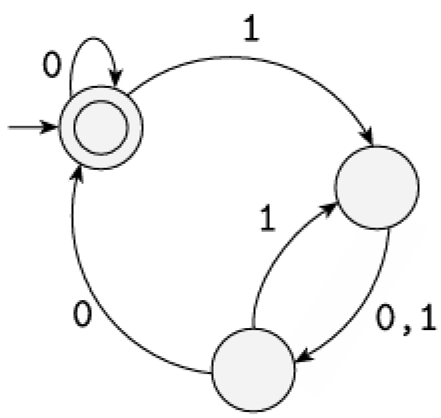
3.16 d) intersect X & Y be 2 Turing recognizable Lang that have Turing machines M_X & M_Y respectively.
Now the intersect of these Lang = denoted by L_{XY} & the Turing machine recogniz Lang = M_{XY}.
input str s L_{XY} M_{XY}:
Turing machine M_X runs s. → If s accepts → M_Y runs on s. else = rejects
· M_Y accepts s → accepted by the Turing machine else = rejects

Conclusion:

thus, the collection of Turing recognizable Lang = closed under intersect operation.

Decidability: 4.1, 4.2, 4.3, 4.4

A **4.1** Answer all parts for the following DFA M and give reasons for your answers.



a. Is $\langle M, 0100\rangle \in A_{\mathsf{DFA}}$?

b. Is $\langle M, 011\rangle \in A_{\mathsf{DFA}}$?

c. Is $\langle M\rangle \in A_{\mathsf{DFA}}$?

d. Is $\langle M, 0100\rangle \in A_{\mathsf{REX}}$?

e. Is $\langle M\rangle \in E_{\mathsf{DFA}}$?

f. Is $\langle M, M\rangle \in EQ_{\mathsf{DFA}}$?

4.1 A) W = On input ⟨M, w⟩, where M is a DFA & w is str.
starting & fin (accepting) state of M is x. After taking the 1st input 0 from str 0100 to the next state still x. Now 1 input to string 0100 the next state = y. Sim taking 4th '0' input 0100 the next state becomes x which is accepted

M does Accept input str 0100.

B) After taking the 1st input 0 from str 011, the next state is still x.
After taking 2nd input 1 from str 011 the next state is y.
Similarly after taking 3rd '0' input from 011 the next state becomes z which isn't accepted

M Does NOT accept input str 011

C) Input str w is not given in A_{DFA}. Here A_{DFA} is not in proper format.

M Does NOT accept Input str.

D) Consider a T.M w which decides whether reg expression can generate provided str or not
w = On input ⟨R, w⟩, where R is a reg expression, w is str.
Here, pt M is a DFA instead of regular expression but for A_{Rex} it should be reg expression.

M cannot generate str 0100

E) ~~E~~ Do T.M which decides E_{DFA}. w = On I⟨M⟩, where M is DFA
mark init state of M, that is x,
repeat state till new state not marked etc. found init & end state are the same that is x. So when applying 1st step of E_{DFA} theorem then init as well as fin state both get marked. It is not acceptable when fin state is marked an empty lan.

M cannot accept any str. M in not ∧

F) T.M w which decides EQ_{DFA}. Here L(M) = L(M)
therefore ~~deriv~~ deriv Lang L(C) becomes empty. Thus T.M w accepts deriv Lang L(C).

yes the Lang of M accepts itself.

4.2 Consider the problem of determining whether a DFA and a regular expression are equivalent. Express this problem as a language and show that it is decidable.

4.2 Express Lang $L = \{R, S\}$ R is a DFA. S is reg expression w/ $L(R) = L(S)$. R decides the lang $EQ_{DFA} = \langle P, Q \rangle$ P & Q are DFA $L(P) = L(Q)$.

· Assume T = T.M which decides Lang L.

·Can be:

T = on input $L = \langle R, S \rangle$ R = DFA S = reg expression.

· Convert R into DFA D_R using algorithm in proof of Kleene Theorem.

Operate T.M as a decider F using theorem 4.5 on input $\langle R, D_S \rangle$

If.

- F accepts, accept the Lang L.
- F rejects, reject the Lang L.

4.3 Let $ALL_{DFA} = \{\langle A \rangle \mid A \text{ is a DFA and } L(A) = \Sigma^*\}$. Show that ALL_{DFA} is decidable.

4.3 We need to show that there is a T.M. that can decide whether a given DFA A accepts all possible strings over it's alphabet Σ ($L(A) = \Sigma^*$).

input: input to the decider is a DFA A

2. Reachable States:
Perform BFS Breadth 1st Search, or DFS starting from initial state of A.
Mark all states that are reachable from S_0 or S_1,
If any state is not reachable, then A does not accept All str. In this case, reject. ($L(A) \neq \Sigma^*$)

3. Check all reachable states are accepting
After identifying the reachable states, check if all these states are accepting states.
If any reachable state is a non-accepting state, then A does not accept All str In this case, reject the input.

4. Accept if All reachable states are Accepting:
If all reachable states are accepting states then A accepts all str, & $L(A) = \Sigma^*$. In this case, accept input.

- The Algo decides whether $L(A) = \Sigma^*$ for a given DFA A. Since the procedure always halts ALL DFA is decidable.

4.4 Let $A\varepsilon_{\mathsf{CFG}} = \{\langle G \rangle \mid G \text{ is a CFG that generates } \varepsilon\}$. Show that $A\varepsilon_{\mathsf{CFG}}$ is decidable.

4.4 Decidability of Lang = $A_{\varepsilon CFG} = \{\langle G \rangle \mid G$ is a CFG that ge[nerates] $\varepsilon\}$

given: In this Lang $A_{\varepsilon CFG}$ is given

Lang $A_{\varepsilon CFG}$ is decidable, built a T.M T for deciding the lang $A_{\varepsilon CFG}$ For all CFG = G.

is The Gram:

- G deriv ε then T(<G>) accepts
- G Doesn't deriv ε then T(<G>) rejects.

For proving the decidability of $A_{\varepsilon CFG}$ Firsty convert the CFG = G into equiv G' in CNF. If $S \to \varepsilon$ is the rule in the CFG G' then it means that G' deriv ε
If the CFG G' deriv ε then G also deriv it as $L(G) = L(G')$. As G' is in CNF only possible ε-rule in G' is $S \to \varepsilon$. If G' contains $S \to \varepsilon$ in production rules then $\varepsilon \in L(G')$. If G' doesn't contain rule $S \to \varepsilon$ then $\varepsilon \notin L(G')$

T. M T = on input <G> where G = CFG

- Convert G = CFG G'
- If G' has the Production Rule $S \to \varepsilon$ then accept. else → reject.

Conclusion: It's clear $\langle G \rangle \in A_{\varepsilon CFG}$ Iff $\langle G, \varepsilon \rangle$ it also belongs to the ACFG. So above construct is correct. $A_{\varepsilon CFG}$ = Language is decidable